

PostalMS

Pseudocode and Algorithm Design

Data Structures | Gmail Validation | Stamp Pricing | City Detection | Stamp Order Flow | Receiver Validation

- This document covers pseudocodes for all three custom data structures and all key business logic algorithms including the new Sprint 4 features.

Algorithms Covered

#	Algorithm	Complexity	Used In
1	Custom Hash Table	O(1) average add, search, delete	Parcel lookup by tracking ID
2	Custom Binary Search Tree	O(log n) average insert, search, delete	Sorted parcel display
3	Custom Queue (FIFO)	O(1) enqueue, dequeue, peek	Delivery assignment processing
4	Gmail Validation	O(n) where n = email length	LoginForm, RegisterForm, StampsView, ParcelsView
5	Stamp Price Calculation	O(1) constant time	StampsView ordering system
6	City Detection	O(c) where c = 7 cities	FindUsView location filtering
7	Stamp Order Flow	O(1) per step	StampsView delivery and collect
8	Receiver Details Validation	O(1) per field check	ParcelsView Step 4 send form

1. Custom Hash Table

1.1 Overview

The CustomHashTable converts a parcel tracking ID into an array index using a polynomial rolling hash function. Average $O(1)$ for add, search and delete.

1.2 Hash Function

```
FUNCTION HashFunction(key):
    hash = 0
    FOR each character c in key:
        hash = (hash * 31) + ASCII value of c
    RETURN absolute value of (hash mod tableSize)
END FUNCTION
```

1.3 Pseudocode

1.3.1 Add

```
FUNCTION Add(key, value):
    index = HashFunction(key)
    currentNode = table[index]
    WHILE currentNode is not null:
        IF currentNode.key equals key:
            currentNode.value = value
            RETURN
        currentNode = currentNode.next
    newNode = new Node(key, value)
    newNode.next = table[index]
    table[index] = newNode
    count = count + 1
    IF (count / tableSize) > 0.75: CALL Resize()
END FUNCTION
```

1.3.2 Search

```
FUNCTION Search(key):
    index = HashFunction(key)
    currentNode = table[index]
    WHILE currentNode is not null:
        IF currentNode.key equals key: RETURN currentNode.value
        currentNode = currentNode.next
    RETURN null
END FUNCTION
```

1.3.3 Delete

```
FUNCTION Delete(key):
    index = HashFunction(key)
    currentNode = table[index]; previousNode = null
    WHILE currentNode is not null:
        IF currentNode.key equals key:
            IF previousNode is null: table[index] = currentNode.next
            ELSE: previousNode.next = currentNode.next
            count = count - 1; RETURN true
        previousNode = currentNode; currentNode = currentNode.next
    RETURN false
END FUNCTION
```

2. Custom Binary Search Tree (BST)

2.1 Overview

The CustomBST stores parcels where every left child has a smaller key and every right child has a larger key. $O(\log n)$ average insert and search, $O(n)$ in-order traversal.

2.2 Pseudocode

2.2.1 Insert

```
FUNCTION InsertRecursive(node, key, value):
    IF node is null: count++; RETURN new BSTNode(key, value)
    IF key < node.key: node.left = InsertRecursive(node.left, key, value)
    ELSE IF key > node.key: node.right = InsertRecursive(node.right, key, value)
    ELSE: node.value = value
    RETURN node
END FUNCTION
```

2.2.2 InOrder Traversal

```
FUNCTION InOrderRecursive(node, result):
    IF node is null: RETURN
    InOrderRecursive(node.left, result)
    result.Add(node.key, node.value)
    InOrderRecursive(node.right, result)
END FUNCTION
```

2.2.3 Delete

```
FUNCTION DeleteRecursive(node, key):
    IF node is null: RETURN null
    IF key < node.key: node.left = DeleteRecursive(node.left, key)
    IF key > node.key: node.right = DeleteRecursive(node.right, key)
    ELSE:
        IF node.left is null: RETURN node.right
        IF node.right is null: RETURN node.left
        successor = FindMin(node.right)
        node.key = successor.key; node.value = successor.value
        node.right = DeleteRecursive(node.right, successor.key)
    RETURN node
END FUNCTION
```

3. Custom Queue (FIFO)

3.1 Overview

The CustomQueue is a singly linked list with front and rear pointers. O(1) for enqueue, dequeue and peek with no worst-case degradation.

3.2 Pseudocode

3.2.1 Enqueue

```
FUNCTION Enqueue(value):
    newNode = new QueueNode(value)
    IF rear is null: front = newNode; rear = newNode
    ELSE: rear.next = newNode; rear = newNode
    count = count + 1
END FUNCTION
```

3.2.2 Dequeue

```
FUNCTION Dequeue():
    IF front is null: THROW InvalidOperationException
    value = front.value
    front = front.next
    IF front is null: rear = null
    count = count - 1
    RETURN value
END FUNCTION
```

4. Gmail Validation (New - Sprint 4)

4.1 Overview

Enforced in LoginForm, RegisterForm, StampsView and ParcelsView. All emails must end with @gmail.com and have a valid username. Added following professor feedback.

4.2 Pseudocode

```
FUNCTION ValidateGmail(email):
    IF email is null or empty: RETURN false
    IF email does not end with '@gmail.com': RETURN false
    IF email length <= length of '@gmail.com': RETURN false
    IF email starts with '@': RETURN false
    RETURN true
END FUNCTION
```

4.3 Applied In

Form	Usage
LoginForm.cs	Validates email before checking credentials. Non-Gmail rejected before any database call.
RegisterForm.cs	Validates email before registering. User sees error if not @gmail.com.

Form	Usage
StampsView.cs	Validates email for delivery and collect order confirmations.
ParcelsView.cs	Validates sender email in Step 4 receiver details.

5. Stamp Price Calculation (New - Sprint 4)

5.1 Overview

Calculates total stamp order price including delivery fee, 20% VAT and 2% service fee. Updates live as user changes stamp type, quantity and delivery method.

5.2 Pseudocode

```
FUNCTION CalcStampTotal(unitPrice, quantity, deliveryFee):
    stampSubtotal = unitPrice * quantity
    subtotal = stampSubtotal + deliveryFee
    vat = Round(subtotal * 0.20, 2)
    serviceFee = Round(subtotal * 0.02, 2)
    total = Round(subtotal + vat + serviceFee, 2)
    RETURN total
END FUNCTION
```

5.3 Delivery Fees

Method	Fee	Delivery Time
Standard Home Delivery	GBP 1.50	2-3 working days
Express Home Delivery	GBP 3.99	Next working day
Click and Collect	Free -- GBP 0.00	From selected date

6. City Detection (New -- Sprint 4)

6.1 Overview

FindUsView auto-detects the user city from the City column in the Users database table. Shows PostalMS locations in that city. Supports 7 UK cities with 76 locations total.

6.2 Pseudocode

```
FUNCTION GetUserCity(userID, database):  
    result = database.Query('SELECT City FROM Users WHERE UserID = userID')  
    IF result has rows AND City column exists:  
        city = result.Rows[0]['City']  
        IF city is not empty: RETURN city  
    result = database.Query('SELECT Address FROM Users WHERE UserID = userID')  
    IF result has rows:  
        address = result.Rows[0]['Address']  
        FOR each city in supportedCities:  
            IF address contains city (case insensitive): RETURN city  
    RETURN 'London'    // Default  
END FUNCTION
```

7. Stamp Order Flow (New -- Sprint 4)

7.1 Overview

Food-app style ordering: choose stamps, select Delivery or Collect, fill details, choose payment.

Delivery needs name, address, city, postcode, email. Collect needs name, phone, DOB, location, date, time slot, email.

7.2 Pseudocode

```
FUNCTION PlaceStampOrder():
    IF stampType is null OR quantity < 1: SHOW error; RETURN
    isDelivery = Delivery card is selected
    IF isDelivery:
        IF name is empty OR address is empty: SHOW error; RETURN
        email = txtDeliveryEmail.Text
    ELSE: // Click and Collect
        IF name is empty: SHOW error; RETURN
        IF DOB is empty or invalid: SHOW error; RETURN
        location = cboPickupLocation.SelectedItem
        date = dtPickup.Value
        timeSlot = cboPickupTime.SelectedItem
        email = txtCollectEmail.Text
    payOnline = Pay Now card is selected
    IF NOT payOnline AND isDelivery:
        SHOW 'Pay In Store only available with Click and Collect'
        RETURN
    IF ValidateGmail(email) is false: SHOW error; RETURN
    deliveryFee = isDelivery ? (isExpress ? 3.99 : 1.50) : 0.00
    total = CalcStampTotal(unitPrice, quantity, deliveryFee)
    orderId = 'STO-' + DateTime.Now.ToString('yyyyMMddHHmmss')
    stampOrders.Add(new StampOrder(orderId, ...))
    SHOW confirmation dialog
    NAVIGATE to My Stamp Orders tab
END FUNCTION
```

7.3 Missed Pickup

```
FUNCTION CheckMissedPickup(order):
    IF order.Method is not 'Click and Collect': RETURN
    IF order.Missed is already true: RETURN
    orderDate = Parse(order.Date)
    IF orderDate + 48 hours < DateTime.Now AND order.Status = 'Processing':
        order.Missed = true
        order.Status = 'Missed -- Rescheduled'
        order.RescheduleDate = DateTime.Now + 2 days
        SHOW red missed banner on order card in My Orders
    END IF
END FUNCTION
```

8. Receiver Details Validation (New -- Sprint 4)

8.1 Overview

ParcelsView Step 4 collects full receiver details before a parcel can be submitted. All required fields are validated. This was added following professor feedback requesting more complete receiver information.

8.2 Fields Collected

Field	Required	Validation Rule
Receiver Full Name	Yes	Must not be empty
Receiver Phone Number	Yes	Must not be empty
Receiver Email Address	No	If provided must be valid email format
Receiver Street Address	Yes	Must not be empty
Receiver City	Yes	Must not be empty
Receiver Postcode	Yes	Must not be empty
Special Delivery Instructions	No	Optional -- placeholder text shown if empty
Sender Email (for confirmation)	Yes	Must be valid @gmail.com address

8.3 Pseudocode

```
FUNCTION ValidateReceiverDetails():
    IF txtRcvName is empty:
        SHOW 'Please enter receiver full name'; RETURN false
    IF txtRcvPhone is empty:
        SHOW 'Please enter receiver phone number'; RETURN false
    IF txtRcvAddr is empty:
        SHOW 'Please enter receiver street address'; RETURN false
    IF txtRcvCity is empty:
        SHOW 'Please enter receiver city'; RETURN false
    IF txtRcvPost is empty:
        SHOW 'Please enter receiver postcode'; RETURN false
    IF ValidateGmail(txtSenderEmail.Text) is false:
        SHOW 'Please enter valid @gmail.com for confirmation'; RETURN false
    RETURN true    // All required fields pass validation
END FUNCTION
```


9. Summary

This document covers all eight algorithms in PostalMS. The three data structures form the core data layer. The five new Sprint 4 algorithms add complete business logic for stamps ordering, city detection, Gmail validation and receiver details validation.

#	Algorithm	Best	Average	Worst
1	Hash Table Add/Search/Delete	O(1)	O(1)	O(n)
2	BST Insert/Search/Delete	O(1)	O(log n)	O(n)
3	BST InOrder Traversal	O(n)	O(n)	O(n)
4	Queue Enqueue/Dequeue/Peek	O(1)	O(1)	O(1)
5	Gmail Validation	O(1)	O(n)	O(n)
6	Stamp Price Calculation	O(1)	O(1)	O(1)
7	City Detection	O(1)	O(c)	O(c)
8	Stamp Order Flow	O(1)	O(1)	O(1)
9	Receiver Details Validation	O(1)	O(1)	O(1)